

How the 11×11 simulation computes spring forces

Exponential springs, step by step (theory and code)

Daniel Miles

March 26, 2026

Abstract

This note explains the *idea* behind the nonlinear spring model used in the blast-loaded sandwich-plate simulations, then shows *where* each piece lives in the Julia program. The core is a simple exponential relation between how far two neighboring masses move *relative to each other* and the force between them. We also spell out how the lattice is wired (nearest neighbors, diagonals, damping), how layer materials enter the vertical and diagonal springs, and a few honest limitations (for example, the bundled potential-energy routine does not always match the forces the solver actually uses).

1 Why an exponential spring?

A linear spring (Hooke’s law) says force is proportional to stretch. Real foams and many materials stiffen more strongly once deformations grow; an exponential law is one way to capture that in a mass–spring network without leaving the familiar picture of nodes connected by springs.

Plain-language picture. Imagine two beads and a link between them. We pick a *reference* layout (a regular grid). While the simulation runs, each bead’s **displacement** is how far it has moved away from its reference spot. The program measures how much the two beads have shifted **relative to each other**; that relative shift sets the link’s stretch proxy d . The push–pull along the link grows like $e^{\alpha d} - 1$ (after a scale k), so small motions feel nearly linear, while larger ones ramp up faster.

2 Reference lattice and what the solver actually stores

2.1 The undisturbed grid

Think of an 11×11 array of masses, rows i and columns j , both running from 1 to N with $N = 11$. In the reference (stress-free layout used only as a bookkeeping frame), the position of mass (i, j) is

$$\mathbf{X}_{ij}^{\text{eq}} = \begin{bmatrix} (j-1)\Delta \\ (i-1)\Delta \end{bmatrix}, \quad \Delta = 1.0 \text{ m.} \quad (1)$$

So neighbors along a row sit Δ apart horizontally; neighbors along a column sit Δ apart vertically; diagonals sit $\Delta\sqrt{2}$ apart in the reference frame.

2.2 Displacements versus “true” positions

The differential equation solver does *not* store full positions $\mathbf{X}_{ij}(t)$ in the half of the state vector devoted to “positions.” It stores **displacements** $\mathbf{u}_{ij}(t)$ from that reference:

$$\mathbf{X}_{ij}(t) = \mathbf{X}_{ij}^{\text{eq}} + \mathbf{u}_{ij}(t). \quad (2)$$

At the start of a run, every mass is at rest at its reference node: $\mathbf{u}_{ij}(0) = \mathbf{0}$. When you see animations, the plotted positions add the reference grid back in.

The reference grid is `create_equilibrium_grid()` using `GRID_SPACING = 1.0`. The state vector `u` begins as `u0 = zeros(...)`; its position block is reshaped into `pos`, with `pos[:,k] = u_ij` for index `k = lattice_idx(i,j)`. Plots use `current = displacement + equilibrium_grid`.

2.3 What enters the spring: relative displacement, not raw distance in space

For a bond between masses a and b , the implementation feeds spring routines the *difference of nodal displacements*:

$$\mathbf{r} = \mathbf{u}_b - \mathbf{u}_a. \quad (3)$$

Let $d = \|\mathbf{r}\|$. When the whole lattice is still at its reference placement, every $\mathbf{u}$ is zero, so $\mathbf{r} = \mathbf{0}$ and $d = 0$: **no spring force**. That matches the idea of starting from an unstressed reference.

How this relates to “elongation.” If two neighbors only slide apart along their bond axis (for example, a horizontal link and only horizontal relative motion), then d agrees with the usual notion of how much longer that link is than in the reference layout. If the motion mixes shearing or both x and y components, d is the length of the *relative displacement vector*; the force points along that vector. That is slightly different from a bar whose tension depends only on the **current straight-line distance** $|\mathbf{X}_b - \mathbf{X}_a|$ in the lab frame. For many load cases the distinction is minor; it matters most when shear between the two nodes is large relative to stretch.

3 Force on one spring: magnitude and direction

Let $\mathbf{r} = \mathbf{u}_b - \mathbf{u}_a$ as in (3) and $d = \|\mathbf{r}\|$. For d above a tiny numerical floor (10^{-12} in code), the model uses

$$\hat{\mathbf{r}} = \frac{\mathbf{r}}{d}, \quad (4)$$

$$F_{\text{mag}} = k \left(e^{\alpha d} - 1 \right), \quad (5)$$

$$\mathbf{F}_{\text{on } a \text{ from } b} = F_{\text{mag}} \hat{\mathbf{r}}. \quad (6)$$

So the magnitude grows with d and blows up faster than linear once αd is order one or larger. The direction is always the line from a ’s displacement state toward b ’s in displacement space. If d is below the floor, the force is set to zero to avoid dividing by zero.

Units. In the project files, k (`K_COUPLING`, etc.) is given in newtons, and α in m^{-1} . That makes $e^{\alpha d} - 1$ dimensionless and F_{mag} a force. The *small-motion* stiffness (§4) is governed by the product $k\alpha$, which has units N/m .

One bond is evaluated in `spring_force_2d(pos1, pos2, k, alpha)`, where `pos1` and `pos2` are the two displacements $\mathbf{u}_a$ and $\mathbf{u}_b$. The force returned is on the first mass from the second. The time-stepper adds these contributions for every neighbor; when the neighbor’s turn comes, the opposite end of the link contributes the opposite force, so pairs stay balanced.

4 Stored energy and the nearly-linear regime

There is a potential energy for one bond that matches the magnitude in (6):

$$U(d) = \frac{k}{\alpha} \left(e^{\alpha d} - \alpha d - 1 \right), \quad d = \|\mathbf{u}_b - \mathbf{u}_a\|. \quad (7)$$

One checks that $dU/dd = k(e^{\alpha d} - 1) = F_{\text{mag}}$. For very small stretches ($\alpha d \ll 1$),

$$e^{\alpha d} - 1 = \alpha d + \frac{1}{2}(\alpha d)^2 + \dots \Rightarrow F_{\text{mag}} \approx k\alpha d.$$

So for tiny relative motion the spring “acts linear” with stiffness about $k\alpha$ in the variable d . That is *not* the same number as a Hooke stiffness tied to real-world length change unless you relate d carefully to geometry (§2).

The same $U(d)$ form appears in `potential_energy_2d_with_diagonals_and_backplate` per bond. **Caution:** for some bond types that helper uses different k, α than the dynamics (see §8), so do not use it alone to audit total energy when columns use different materials.

5 Wiring the whole lattice: neighbors, layers, and dashpots

5.1 Who is connected to whom?

Each interior mass talks to up to eight neighbors: four along the grid (left, right, up, down) and four diagonals. Edge masses have fewer connections. This is the same connectivity described in the sandwich-plate write-up.

5.2 Which springs are “softer” or “stiffer” by design?

- **Horizontal links.** Neighbors in the same row (left–right) share one global ($K_{\text{COUPLING}}, \alpha_{\text{COUPLING}}$) and one dashpot strength C_{DAMPING} . They ignore the column material recipe.
- **Vertical links** (same column, adjacent rows) use the material assigned to that column: stiffness k , rate α , and damping c from the active material index for column j .
- **Diagonals** use the diagonal stiffness and α from the column of the mass that is currently being updated (the code walks each mass and visits its diagonal neighbors).
- **Dashpots** act only on the four nearest-neighbor springs, not on the diagonals.

Bond	Stiffness (k, α)	Damping
Horizontal	$K_{\text{COUPLING}}, \alpha_{\text{COUPLING}}$ everywhere	C_{DAMPING}
Vertical	Column j : material lookups <code>get_column_k_coupling,</code> <code>get_column_alpha_coupling</code>	column c via <code>get_column_c_damping</code>
Diagonal	Column j of current node: <code>get_column_k_diagonal,</code> <code>get_column_alpha_diagonal</code>	none

Table 1: Bond parameters in the ODE right-hand side (`lattice_2d_rhs...`).

5.3 Default material scaling

The program builds $N = 11$ material records. Material 1 uses the base numbers; for index $m > 1$, stiffnesses and damping scale by σ^{m-1} with $\sigma = \text{MATERIAL_MULTIPLIER}$ (default 2/3), while α is usually kept at the base value. A `material_order` permutation places those materials in columns for optimization studies.

Material tuples are built in `create_default_material_properties`. During integration, parameters come from `p.material_order` and `p.materials` when present; otherwise legacy column scaling via `MATERIAL_MULTIPLIER` is used for vertical springs. The ODE right-hand side is `lattice_2d_rhs_with_diagonals_and_backplate!`.

5.4 Viscous damping (nearest neighbors only)

Between nearest neighbors, an extra piece proportional to relative velocity is added on mass a :

$$\mathbf{F}_{\text{damp}} = -c(\mathbf{v}_a - \mathbf{v}_b), \quad (8)$$

with c taken from the same routing rules as the vertical spring for column neighbors (or globals for horizontal neighbors).

6 Forces that are not springs

These are separate from the exponential links but appear in the same acceleration routine:

- **Backplate.** Rightmost column: if absolute x passes the plate, a stiff spring-like penalty (and damping on v_x pushing into the wall) pushes back.
- **Blast-type load.** For a short active window, trapezoidally distributed forces drive the left edge (column 1).

Wall logic uses absolute $x = (N-1)*\text{GRID_SPACING} + \text{pos}_k[1]$. Loads use `F_ACTIVE_TIME`, `F_MAG`, and row weights `get_distributed_force_magnitude`.

7 Default numerical values in the simulation

Table 2 lists the global constants as in the current source file. Vertical and diagonal springs felt by a given column also depend on which material index sits in that column after scaling.

8 A limitation to know about: potential energy helper vs. real forces

The routine `potential_energy_2d_with_diagonals_and_backplate` is convenient, but it does *not* always use the same (k, α) on every bond that the ODE uses. In particular, vertical springs in that helper use uniform K_{COUPLING} and α_{COUPLING} for all columns, while the dynamics apply column-specific materials to vertical links. Diagonal terms in the helper also do not mirror the column-wise diagonal (k, α) used in the right-hand side.

Name / symbol	Value	Role
N	11	Grid side length
Δ (GRID_SPACING)	1.0 m	Reference spacing
m (MASS)	1.0 kg	Mass per node
K_{COUPLING}	100 N	Horizontal NN springs; vertical scale reference
α_{COUPLING}	10 m^{-1}	NN exponential rate (code comments say “decay” but here $\alpha > 0$ <i>stiffens</i> with stretch)
K_{DIAGONAL}	50 N	Diagonal scale (half of coupling in default)
α_{DIAGONAL}	10 m^{-1}	Diagonal rate
C_{DAMPING}	5 N s m^{-1}	Base NN dashpot
σ (MATERIAL_MULTIPLIER)	2/3	Softens k, c for higher material indices
$k_{\text{wall}}, c_{\text{wall}}$	$10\,000 \text{ N m}^{-1}$, 10 N s m^{-1}	Backplate
d_{back}	0.001 m	Offset to plate
$F_{\text{tot}}, t_{\text{act}}$	1200 N, 0.15 s	Edge load budget and duration

Table 2: Global parameters (const values in `src/lattice_simulation_11x11.jl`).

Takeaway. Trust `lattice_2d_rhs_with_diagonals_and_backplate!` for what the simulation actually does; use the energy function with care when layers are permuted or material arrays differ from the default pattern.

9 Matching symbols in the written paper to this program

The manuscript writes spring forces using something like $\mathbf{r} = \mathbf{x}_{i',j'} - \mathbf{x}_{i,j}$. In the code, $\mathbf{x}$ must mean the **same displacements** stored in `pos`, not absolute lab coordinates $\mathbf{X}$. Starting from $\mathbf{u} = \mathbf{0}$ is the same as starting from $\mathbf{X} = \mathbf{X}^{\text{eq}}$ in the paper. If you accidentally plugged absolute positions into the same formula, every bond would appear stretched at $t = 0$ and the model would predict huge initial forces, which is not what the program does.

Appendix: core routine (Julia)

The following is the literal exponential spring used between two displacement vectors:

```

1 function spring_force_2d(pos1, pos2, k, alpha)
2     displacement = pos2 - pos1
3     distance = norm(displacement)
4     if distance < 1e-12
5         return zeros(2)
6     end
7     direction = displacement / distance
8     force_magnitude = k * (exp(alpha * distance) - 1.0)
9     return force_magnitude * direction
10 end

```